PneumoScan Technical Report
BSc. in Software Engineering
Mugabo Patricie

Repository: github.com/pmugabo/pneumoscan

Supervisor: Emmanuel Adjei

October 22nd, 2026

1. Scope alignment and feature validation

PneumoScan is a research prototype that classifies paediatric chest X-rays into four categories: CLEARED, FLAGGED_URGENT, ESCALATED (to a specialist) and REJECTED, based on a calibrated confidence and Mahalanobis out-of-distribution (OOD) score. The table below shows a comparison between the approved proposal (v7) and what is currently in the repository. "Evidence" is the file or result that provides evidence.

Planned feature	Intended purpose	Status	Evidence
Image upload and validation	Accept JPEG/PNG up to 10 MB; refuse unreadable files	Completed	backend/main.py predict (400 on bad type, size or content); API tests
Pneumonia classifier (7 models compared)	Normal vs pneumonia; baselines justify the choice	Completed	results/model_comparison.csv; EfficientNetB0 deployed (AUC 0.965, test n = 618)
Probability calibration	Confidence that can be compared with a threshold	Completed	Temperature T = 1.293; ECE 0.0077 → 0.0058 (models/gate_params.json)
OOD gate (Mahalanobis)	Refuse inputs unlike the training images	Completed; formal OOD evaluation partial	gate_stats.npz; 4 non-radiograph inputs rejected (Section 3); adult and degraded sets not yet run
Four triage states and gate rule	Replace "always answer" with decide / escalate / reject	Completed	backend/inference.py Engine.predict; docs/ui-*.png
Grad-CAM	Show where the	Completed	results/gradcam_samples.png; known

heatmap	model looked	(not shown for REJECTED)	limitation: some maps weight image margins
Authentication and roles	Clinician, specialist and admin access	Completed	Firebase Authentication; require(*roles) in main.py; firestore.rules deny browser access
History and outcome feedback	Record the confirmed outcome per case	Completed	/api/feedback, /api/cases; frontend/assets/clinician.js
Specialist escalation workflow	Give ESCALATED cases a destination	Added beyond proposal	/api/escalations, notifications, specialist.js. Added because an escalation with nobody to receive it only moved the work.
PDF report and admin monitoring	Audit trail and case reports	Completed	backend/report.py; /api/report/{id}, /api/admin/* (audit log, monitoring)
Versioned model artefact (immutable record)	Reproduce any decision from its model version	Partial	Each prediction stores model_version and the files travel together; no immutable model-version table yet
Patient-level split, manifest, cross-fitted gate	No leakage; separate selection and calibration	Partial	v2 notebook written (pneumoscan-notebook-v2.ipynb); the deployed gate comes from v1 (single validation set, patient-level split)
Usability study with participants	Measure usability and appropriate reliance	Not started	Ethics application prepared for review; sessions only after approval
Hosted deployment, ONNX, offline use	Remote testers; on-device inference	Not implemented	Runs locally with Firebase. Justification: zero budget and ethics gating; planned next

Scope changes and justification. (1) The proposal is named SQLite/PostgreSQL; the prototype uses Cloud Firestore with Firebase Authentication since both are managed, free at this scale and don't require a database server to be run. (2) Specialist routing was added because an ESCALATED result with nobody to receive it only moves the work; the specialist now receives the case, the heatmap and the AI reading together. (3) Evaluation claims are limited to retrospective public data; clinical benefit is not claimed.

2. Technical functionality

Architecture. Browser (plain HTML/JS, role-specific modules) → FastAPI → Engine (PyTorch) → Firestore. The browser signs in with Firebase and sends the ID token with every request; the backend verifies it, checks the role, runs inference and stores the result. Firestore collections: users, patients, predictions, escalations, notifications, audit_logs. Images are stored as downscaled JPEG data URLs (max 768 px) to stay under Firestore's 1 MB document limit; heatmaps are base64 PNG.

Decision logic (the gate). For image x the model outputs a logit z. Calibrated probability p = sigmoid(z/T); confidence c = max(p, 1 − p); distance d = min over class means μ_c of $(f - \mu_c)^T \Sigma^{-1} (f - \mu_c)$, where f is the 1,280-dimensional pooled feature vector and Σ^{-1} a shared precision matrix. Rule: if d > τd (2,849, the value retaining 95% of validation images) → REJECTED; else if c < τc (0.9755) → ESCALATED; else FLAGGED_URGENT if p ≥ 0.5, otherwise CLEARED. Grad-CAM weights the last feature maps by their mean gradients, applies ReLU and normalises to 0–1. Thresholds and statistics are loaded from files exported by the training notebook, so a model cannot be served without its operating point.

Data handling and security. Uploads are checked for type, size and decodability before inference; only clinicians and admins can predict; specialists see only escalations assigned to them; every prediction, feedback and escalation action writes an audit-log entry; firestore.rules block direct browser access, so only the backend reads and writes data. Training used patient-level splits and MD5 duplicate removal (32 duplicate files removed).

Results on the held-out test set (618 images, never used for training, selection or thresholds). Ungated: (all 618 images, calibrated cut-off 0.9755, not 0.5): sensitivity 0.946, specificity 0.870, AUC 0.965. Gate: 457 accepted, 147 escalated (low confidence), 14 rejected (OOD); 73.9% resolved without escalation; on accepted cases sensitivity 1.000 and specificity 0.706. The gate buys sensitivity at the cost of specificity, which is the intended trade for screening. These numbers come from the first notebook and will be replaced by the v2 results.



Figure 1. Flagged-urgent result with Grad-CAM (left) and a non-radiograph rejected by the gate (right).

3. Testing strategies and results

3.1 Unit testing (inference engine). Why: the gate is the safety-critical logic and must be testable without Firebase. How: tests/test_inference.py checks engine construction, DEMO mode when model files are missing (state ESCALATED, no clinical output), preprocessing shape (1×3×224×224), Mahalanobis return type and the result dataclass. Result: 8 of 8 passed (pytest 9.1.1, Python 3.13). Design benefit: DEMO mode means a

missing model file produces a refusal, never a guess.

3.2 Integration testing (API against the Firebase emulators). Why: roles, escalation and audit behaviour cross backend and database. How: tests/test_api.py (19 tests) creates real emulator users and exercises token rejection, role checks, invalid type and oversize uploads, feedback, report, the clinician → specialist → admin escalation workflow and notifications. Result:all 19 tests passed in 4.97 s under firebase emulators:exec (Python 3.9.6, macOS, Apple silicon). Covered: sign-up is forced to the clinician role, so self-registration cannot create an admin.

3.3 Functional testing with different inputs. Why: the system's purpose is correct behaviour on unfamiliar input. How: scripts/benchmark.py sends the sample radiographs and synthetic non-radiographs through the engine (CPU, run on 20 Oct 2026):

Input	**p(pneumonia)**	**Confidence**	**Distance d (τd = 2,849)**	**State**
Normal radiograph IM-0345	0.010	0.990	1,559	CLEARED
Normal radiograph IM-0046	0.104	0.896	2,531	ESCALATED (c < 0.9755)
Pneumonia radiograph person1664	1.000	1.000	1,273	FLAGGED_URGENT
non-xray.png (colour image)	1.000	1.000	89,370,642	REJECTED
Random noise	1.000	1.000	2,595,130,237	REJECTED
Blank white image	0.671	0.671	4,706	REJECTED
Smooth colour gradient	0.957	0.957	7,072	REJECTED

Result: all four non-radiographs were rejected and the three radiographs behaved as expected. The classifier alone gave 100% pneumonia probability for the colour image, which is the reason the OOD gate exists. Finding: blank and gradient images exceed τd by only 1.7× and 2.5×, a thin margin, so an image-quality check (contrast, blank detection) is the next improvement. A normal radiograph (IM-0046) is escalated at confidence 0.896, showing the cost in specificity. The adult, degraded and non-chest test sets in the proposal have not been run yet.

3.4 Performance and compatibility. How: 50 timed calls of Engine.predict after 3 warm-up calls, covering preprocessing, calibrated prediction, distance score, Grad-CAM and heatmap rendering (HTTP and Firestore excluded). Result on a 2-vCPU Intel Xeon 2.8 GHz VM with 7 GB RAM, no GPU, PyTorch 2.14: median **0.30 s**, 95th percentile 0.36 s, maximum 0.39 s, peak memory about 0.8 GB. This meets the 5-second requirement with a wide margin. Improvement identified: the backward pass for Grad-CAM currently runs even when the image will be rejected; skipping it for REJECTED images would save time.

Environment	**Software**	**Result**
Cloud VM, 2 vCPU, Linux (no GPU)	Python 3.13, PyTorch 2.14	8 unit tests and 4 database-free API tests passed (12); median latency 0.30 s
Author laptop (Apple silicon, macOS)	Python 3.9.6 (README)	8 unit and 19 API tests passed; median latency 0.41 s, p95 0.43 s, peak memory 476 MB (Apple silicon, macOS, PyTorch 2.4.1, CPU only)

Browsers	Chrome / Safari	all portals, login, triage, referral and PDF download worked.

4. Code quality, OOP and algorithms

Structure and separation of concerns. backend/inference.py (model and gate), backend/firebase.py (the only module that touches the Firebase SDK), backend/report.py (PDF), backend/main.py (HTTP layer), frontend/assets/ (shared core.js and app.js plus one module per role), scripts/ (seeding, benchmark), notebooks/ (training), tests/, models/, results/. About 1,300 lines of backend Python, 420 lines of tests and 1,175 lines of JavaScript.

OOP. Encapsulation: the Engine class owns the model, gate parameters and statistics; callers use only predict() and info(). Abstraction and data transfer: TriageResult is a dataclass with to_dict(), so the API never builds ad-hoc dictionaries; Pydantic models (Feedback, PatientCreate, EscalationResolve and others) validate request bodies. Dependency injection: Engine(model_dir) accepts a directory, which let the tests build a DEMO engine; require(*roles) is a dependency factory that adds role-based access control to any endpoint. Polymorphism/open–closed: architectures are chosen by _build(arch); adding a model means adding one branch.

Algorithms. Temperature scaling (one-parameter log-loss fit), Mahalanobis distance with a shared precision matrix, percentile threshold fitting, Grad-CAM, MD5 duplicate removal and patient-level splitting, an urgent-first-then-oldest escalation queue, and the autonomy-rate and per-state aggregates behind the admin monitoring page.

Honest limitations. main.py (859 lines) still holds all routes and Firestore access; splitting it into APIRouter modules (auth, predictions, patients, escalations, admin) and a repository layer is the main refactoring planned. _build and _embed use if-chains where a registry of architecture classes would be cleaner. on_event("startup") is deprecated in FastAPI and should become a lifespan handler. The commit history is still short and later work will be committed in small, meaningful steps.

5. Conclusion and next steps

The core product (calibrated classification, OOD gate, four triage states, Grad-CAM, roles, escalation to a specialist, reports and audit) is implemented and runs end to end; the gate rejects every non-radiograph tested and one prediction takes about 0.3 s on CPU. Remaining work, in order: run the v2 notebook (separate selection, calibration and test sets) and replace the model and gate files; run the adult, degraded and non-chest OOD sets; add an image-quality check; refactor main.py into routers; and run the usability sessions after ethics approval. The model was trained on one hospital’s paediatric images, so no claim is made about performance on Rwandan patients or about clinical safety.